Lecture # 2 Database System Concepts and Architecture

Rashmi Dutta Baruah

Department of Computer Science & Engineering

भारतीय प्रौद्योगिकी संस्थान गुवाहाटी

Indian Institute of Technology Guwahati

Guwahati - 781039, INDIA

Outline

- Data Models
- Three-schema architecture
- Data independence
- DBMS Language
- DBMS components
- DBMS architecture

Data Models

- **Data Model**: A set of concepts to describe the *structure* of a database, and certain *constraints* that the database should obey – provides means to achieve abstraction.
- **Data Model Operations**: Operations for specifying database retrievals and updates by referring to the concepts of the data model. Operations on the data model may include *basic operations* and *user-defined operations*.

Categories of data models

- **Conceptual** (**high-level**, **semantic**) data models: Provide concepts that are close to the way many users *perceive* data. (Also called **entity-based** or **object-based** data models.)
- **Physical** (**low-level**, **internal**) data models: Provide concepts that describe details of how data is stored in the computer.
- **Implementation** (**representational**) data models: Provide concepts that fall between the above two, balancing user views with some computer storage details.

Schemas versus Instances

- **Database Schema**: The *description* of a database. Includes descriptions of the database structure and the constraints that should hold on the database.
- **Schema Diagram**: A diagrammatic display of (some aspects of) a database schema.
- **Schema Construct**: A component of the schema or an object within the schema, e.g., STUDENT, COURSE.
- **Database Instance**: The actual data stored in a database at a *particular moment in time*. Also called **database state** (or **occurrence**).

Schema Diagram Example

STUDENT

Name	Roll_Number	Year	Department

COURSE

Course_Name	Course_Number	Credit	Department

Database Schema Vs. Database State

- **Database State:** Refers to the content of a database at a moment in time.
- **Initial Database State:** Refers to the database when it is loaded
- **Valid State:** A state that satisfies the structure and constraints of the database.
- **Distinction**
  - The **database schema** changes *very infrequently*. The **database state** changes *every time the database is updated.*
  - **Schema** is also called **intension**, whereas **state** is called **extension**.

Three-Schema Architecture

- Proposed to support DBMS characteristics of:
  - **Program-data independence**.
  - Support of **multiple views** of the data.
- Defines DBMS schemas at *three levels*:
  - **Internal schema** at the internal level to describe physical storage structures and access paths. Typically uses a *physical* data model.
  - **Conceptual schema** at the conceptual level to describe the structure and constraints for the *whole* database for a community of users. Uses a *conceptual* or an *implementation* data model.
  - **External schemas** at the external level to describe the various user views. Usually uses the same data model as the conceptual level.

Three Schema architecture

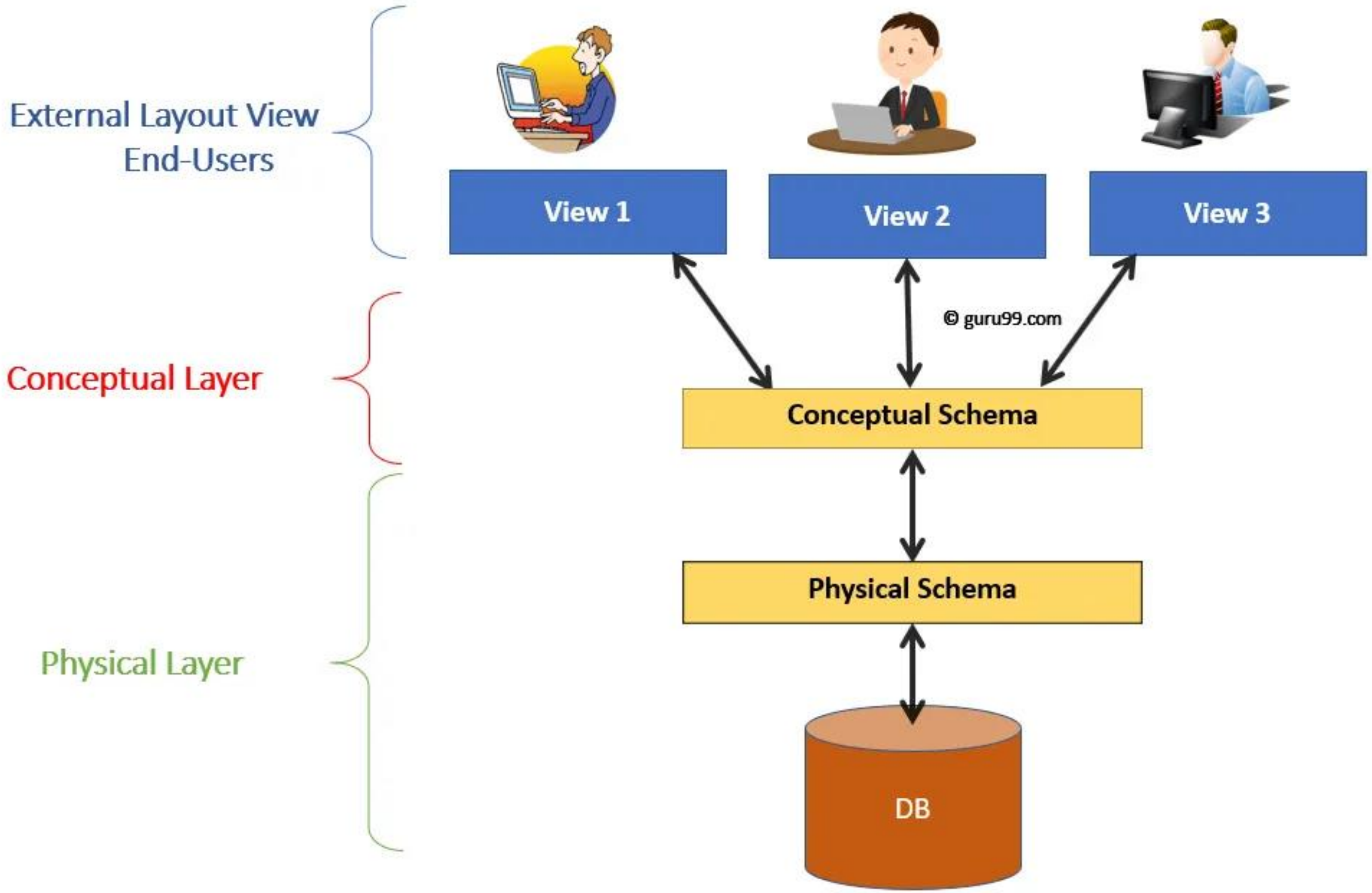


Image source: https://medium.com/nixis-institute/dbms-three-level-architecture-c96ba7a21f5b

Data Independence

- **Logical Data Independence**: The capacity to change the conceptual schema without having to change the external schemas and their application programs.

- **Physical Data Independence**: The capacity to change the internal schema without having to change the conceptual schema.

Data Independence

- **Mappings** among schema levels are needed to transform requests and data. Programs refer to an external schema, and are mapped by the DBMS to the internal schema for execution.

- When a schema at a lower level is changed, only the **mappings** between this schema and higher-level schemas need to be changed in a DBMS that fully supports data independence. The higher-level schemas themselves are *unchanged*. Hence, the application programs need not be changed since they refer to the external schemas.

DBMS Languages

- **Data Definition Language (DDL)**: Used by the DBA and database designers to specify the *conceptual schema* of a database. In many DBMSs, the DDL is also used to define internal and external schemas (views). In some DBMSs, separate **storage definition language** (**SDL**) and **view definition language** (**VDL**) are used to define internal and external schemas.

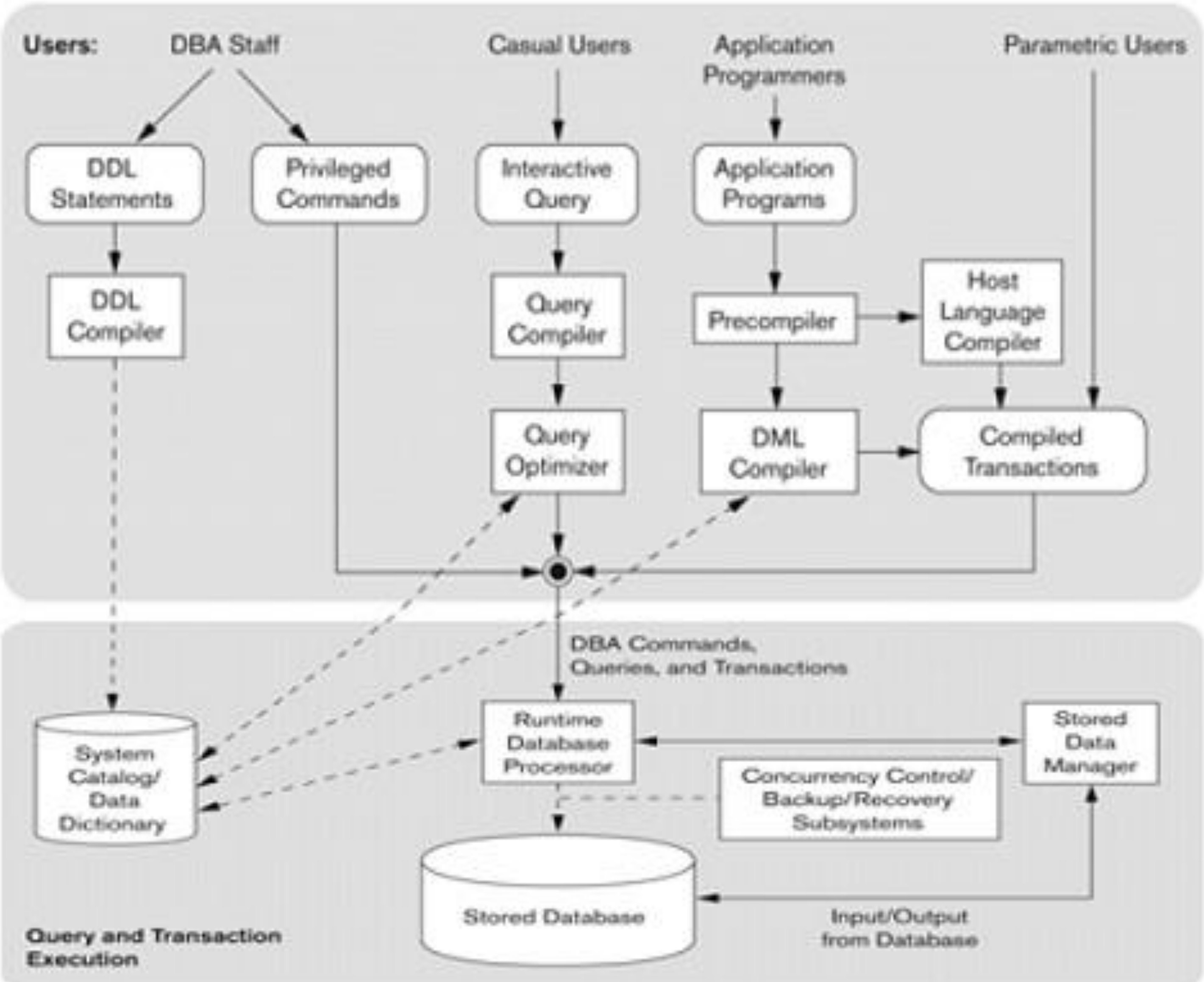
```
CREATE TABLE student (
    s_id INT PRIMARY KEY,
    name VARCHAR(100),
    dept VARCHAR(50),
    class INT,
    INDEX (department)
) ENGINE = InnoDB;
```

```
CREATE TABLE student (
    s_id INT PRIMARY KEY,
    name VARCHAR(100) NOT
NULL,
    dept_id INT,
    FOREIGN KEY
(department_id) REFERENCES
departments(dept_id)
);
```

DBMS Languages

- **Data Manipulation Language (DML)**: Used to specify database retrievals and updates.
  - DML commands (**data sublanguage**) can be *embedded* in a general-purpose programming language (**host language**), such as COBOL, C or an Assembly Language.
  - Alternatively, *stand-alone* DML commands can be applied directly (**query language**).
- In current DBMSs, the mentioned languages are usually not considered distinct languages.

DBMS Component Modules



DBMS Architectures: Centralized and Client-Server

- **Centralized DBMS:** combines everything into single system including- DBMS software, hardware, application programs and user interface processing software.
- **Basic Client-Server Architectures**
  - Clients- user machine provides users interface capabilities and local processing
  - DBMS Server
    - Provides database query and transaction services to the clients
    - Sometimes called query and transaction servers

Two Tier Client-Server Architecture

- **User Interface Programs and Application Programs** run on the client side
- Interface called **ODBC (Open Database Connectivity)** provides an Application program interface (API) allow client side programs to call the DBMS. Most DBMS vendors provide ODBC drivers.

Three Tier Client-Server Architecture

- Common for **Web applications**
- Intermediate Layer called **Application Server** or **Web Server:**
  - stores the web connectivity software and **the rules and business logic (constraints)** part of the application used to access the right amount of data from the database server
  - acts like a conduit for sending partially processed data between the database server and the client.
- **Additional Features- Security:**
  - encrypt the data at the server before transmission
  - decrypt data at the client

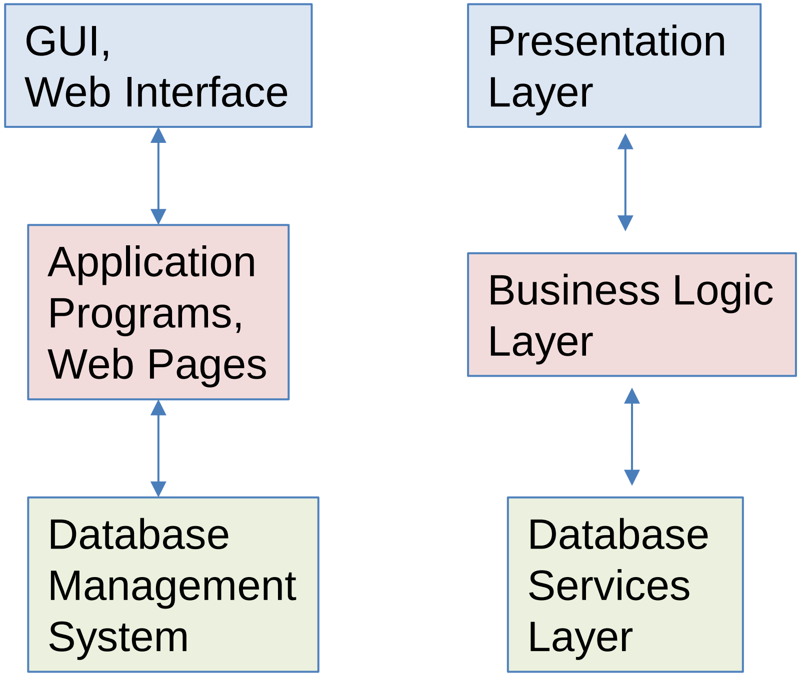


Figure: 3-tier client server architecture

Summary

- Data models and types of data models
- Three-schema architecture and Data independence
- Languages that DBMS support
- DBMS components and 2-tier and 3-tier client server architecture